

Reviewer Pack — Minimal Symplectic Area of Quantum States Bounds Communicati...

Entry cdf70115aa6c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 00:27:59 UTC

Minimal Symplectic Area of Quantum States Bounds Communication Complexity for XOR

Entry ID: cdf70115aa6c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 00:27:59 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum States) **Field B** (complexity object): Boolean Function Complexity

Statement:

For every n -input Boolean function f with communication complexity $\kappa(f)$ for the XOR gate, there exists a quantum state ρ such that the minimal symplectic area of ρ is at most $\kappa(f) * \log(1/2)$. Equivalently: $\kappa(f) = \Theta(\log(2/\det(\rho)))$.

Rationale (proposer's reasoning):

Quantum states are known to encode classical information. The symplectic area of a quantum state might capture the inherent complexity in communicating information classically, reflecting the communication complexity of the XOR gate for that function. By studying symplectic geometry in the context of quantum states, we may uncover new insights into the fundamental nature of classical computation.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b5df660381443741

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient (ρ) between the measured minimal symplectic area of quantum states and the known communication complexity $\kappa(f)$ for XOR gates is $\rho \geq 0.7$, with p -value ≤ 0.05 across at least 100 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum state" AND "communication complexity" AND "Boolean function" - "symplectic area" AND "communication complexity" AND "XOR gate" - "minimal symplectic area" AND "Boolean function complexity" AND "quantum information theory"

Top relevant hits considered: - [s2:f20a88ba0e49b27e8c1c2a823a020085076654ee] Linköping Studies in Science and Technology an On-chip Network Architecture for Hard Real Time Systems an On-chip Networ

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity(f):
        n = len(f)
        if n <= 1:
            return 0
        kappa_f = 0
        for x in range(2**(n-1)):
            y = f[x] ^ f[x + (1 << (n-1))]
            if y != f[x]:
                kappa_f += 1
        return kappa_f

    def symplectic_area(rho):
        n = len(rho)
        det_rho = determinant(rho)
        if det_rho == 0:
            return float('inf')
        return -2 * math.log(det_rho)

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
```

```

det = 0
for j in range(n):
    submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
    det += ((-1) ** j) * matrix[0][j] * determinant(submatrix)
return det

def generate_random_function(n):
    return [random.randint(0, 1) for _ in range(2**n)]

def generate_random_density_matrix(n):
    rho = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        rho[i][i] = Fraction(1, 2)
    return rho

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    kappa_f_total = 0
    symplectic_area_total = 0
    instances_tested = 0

    for _ in range(50):
        f = generate_random_function(n)
        rho = generate_random_density_matrix(n)

        kappa_f = communication_complexity(f)
        area = symplectic_area(rho)

        kappa_f_total += kappa_f
        symplectic_area_total += area
        instances_tested += 1

    if instances_tested == 0:
        continue

    mean_kappa_f = kappa_f_total / instances_tested
    mean_area = symplectic_area_total / instances_tested

    results.append({
        "n": n,
        "mean_kappa_f": mean_kappa_f,
        "mean_area": mean_area,
        "instances_tested": instances_tested
    })

if not results:
    return {
        "metric_name": "symplectic_area",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "no_results"
    }

```

```

n_max = max(result["n"] for result in results)
if n_max < 16:
    return {
        "metric_name": "symplectic_area",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "n_max_too_low"
    }

kappa_f_values = [result["mean_kappa_f"] for result in results]
area_values = [result["mean_area"] for result in results]

def spearman_correlation(x, y):
    n = len(x)
    rank_x = {v: i+1 for i, v in enumerate(sorted(set(x), reverse=True))}
    rank_y = {v: i+1 for i, v in enumerate(sorted(set(y), reverse=True))}

    sum_diff_rank_squared = sum((rank_x[x[i]] - rank_y[y[i]]) ** 2 for i in range(n))
    rho = 1 - (6 * sum_diff_rank_squared) / (n * (n**2 - 1))
    return rho

rho = spearman_correlation(kappa_f_values, area_values)

return {
    "metric_name": "symplectic_area",
    "metric_value": rho,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": rho >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
        sys.exit(0)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho} std=NA support_fraction={support_fraction}")
    else:

```

```

first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con]
print(f"RESULT: FALSIFIED counterexample='rho<{mean_rho}' first_failing_seed={first_failing

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3831468.py", line 140, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3831468.py", line 69, in run_trial
    kappa_f = communication_complexity(f)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3831468.py", line 26, in communication
    y = f[x] ^ f[x + (1 << (n-1))]
          ^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17565
2	preregistration	ollama_remote	glm4:latest	0	0	9448
3	novelty	ollama_remote	glm4:latest	0	0	17621
4	novelty	ollama_remote	glm4:latest	0	0	17042
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15989
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17722
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11658
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21646
9	judge	ollama_remote	glm4:latest	0	0	8616

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137306 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/cdf70115aa6c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/cdf70115aa6c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/cdf70115aa6c.tar.gz (if generated)